

Most Important and Commonly used Pandas function

Page No.:

Date: / /

The two primary data structures in Pandas are the Series (a one-dimensional labeled array) and the DataFrame (a two-dimensional labeled data structure with columns of potentially different types.) You'll always import Pandas using the alias `pd`.

```
import pandas as pd
import numpy as np # Pandas often work well with Numpy
```

I/O and Data Creation

These functions are used to read data from various file formats into a DataFrame and to create test objects.

- `pd.read_csv()` → Reads a comma-separated values (CSV) file into a DataFrame.
eg. `df = pd.read_csv('my-data.csv')`
- `pd.read_excel()` → Reads an Excel file into a DataFrame.
eg. `df = pd.read_excel('my-data.xlsx', sheet_name='sheet1')`
- `pd.read_json()` → Reads a JSON string or file into a DataFrame.
eg. `df = pd.read_json('my-data.json')`
- `pd.read_sql()` → Reads a SQL query or database table into a dataframe.
eg. `df = pd.read_sql('SELECT * FROM table', connection-object)`

- `pd.read_html()` → Reads HTML tables from a URL, string, or file into a list of DataFrames.
eg. `tables = pd.read_html('https://example.com')`
- `pd.DataFrame()` → Creates a DataFrame from various inputs like a dictionary, list or Numpy array.
eg. `df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})`
- `pd.Series()` → Creates a series from a list or array.
eg. `S = pd.Series([10, 20, 30])`
- `df.to_csv()` → Writes DataFrame to a CSV file.
eg. `df.to_csv('output.csv', index=False)`
- `df.to_excel()` → Writes DataFrame to an Excel file.
eg. `df.to_excel('output.xlsx')`

Data inspection & exploration

These methods are essential for getting a quick overview of your data's structure, content, and basic statistics

- `df.head(n)` → Returns the first `n` rows of the DataFrame (default is 5).
eg. `df.head(10)`
- `df.tail(n)` → Returns the last `n` rows of the DataFrame (default is 5).
eg. `df.tail()`

• `df.info()` → Print a concise summary of the DataFrame including column dtypes, non-null counts, and memory usage.
eg. `df.info()`

• `df.shape` → Returns a tuple representing the dimensionality (rows, columns) of the DataFrame.
eg. `df.shape` → (100, 5)

• `df.describe()` → Generates descriptive statistics (count, mean, std, min, quartiles, max) for numerical columns.
eg. `df.describe()`

• `df.dtypes` → Returns the data types of each column.
eg. `df.dtypes`

• `df.columns` → Returns the index (column labels) of the DataFrame.
eg. `df.columns`

• `df.value_counts()` → For a Series, counts the frequency of each unique value.
eg. `df['Country'].value_counts()`

Data Selection and Indexing

Pandas provides powerful methods for selecting and filtering data based on position, labels or conditions.

• `df['column_name']` → Select a single column, returning a Series.

eg. `df['age']`

• `df[['col1', 'col2']]` → Selects multiple columns, returning a DataFrame.

eg. `df[['name', 'age']]`

• `df.loc[row_labels, col_labels]` → Selects data by label (axis labels). Can use slices and lists.

eg. `df.loc[10:20, ['name', 'age']]`
`df.loc[df['age'] > 30]`

• `df.iloc[row-indices, col-indices]` → Selects data by integer position (Zero-based).

eg. `df.iloc[0:5, 0:3]`
`df.iloc[-1]`

• `df.at[row_label, col_label]` → Accesses a single value by label (faster than `.loc` for scalars).

eg. `df.at[0, 'age']`

• `df.iat[row_index, col_index]` → Accesses a single value by integer position (faster than `.iloc` for scalars).

eg. `df.iat[1, 2]`

• `df.query(expression)` → Selects rows based on a query expression string.

eg. `df.query('age > 30 and city == "New York"')`

• Boolean Indexing → Uses a boolean Series to filter rows based on a condition
 eg. `df[df['age'] > 30]`

• Boolean Operators → Combine multiple conditions with & (and), | (or), ~ (not). Ensure each condition is in parentheses
 eg. `df[(df['age'] > 30) & (df['salary'] < 50000)]`

Data cleaning and transformation

Real-world data is often messy. These functions help handle missing values, duplicates, and data type issues.

• `df.isnull()` → Detects missing values (returns a boolean DataFrame of the same shape)
 eg. `df.isnull().sum()`

• `df.notnull()` → Detects non-missing values.
 eg. `df[df['age'].notnull()]`

• `df.dropna()` → Drops rows (or columns with `axis=1`) that contain missing values.
 eg. `df.dropna()`
`df.dropna(subset=['name', 'age'])`

• `df.fillna()` → Fills missing values with a specified value or a computed value (e.g., mean)
 eg. `df['age'].fillna(df['age'].mean())`
`df.fillna({'age': 0, 'name': 'Unknown'})`

- `df.duplicated()` → Returns a boolean series indicating duplicate rows.
eg. `df[df.duplicated()]`
- `df.drop_duplicates()` → Removes duplicate rows.
eg. `df.drop_duplicates(subset=['user_id'])`
- `df.astype()` → Casts a column to a specified data type.
eg. `df['age'] = df['age'].astype(float)`
- `df.replace()` → Replaces values in a Series or DataFrame.
eg. `df['gender'].replace({'M': 'Male', 'F': 'Female'})`
- `df.rename()` → Renames columns or index labels
eg. `df.rename(columns={'old_name': 'new_name'})`
- `df.set_index()` → Sets a column as the index of the DataFrame.
eg. `df.set_index('user-id', inplace=True)`
- `df.sort_values()` → Sorts the DataFrame by one or more columns.
eg. `df.sort_values('age', ascending=False)`
- `df.apply()` → Applies a function along an axis of the DataFrame or to a Series.
eg. `df['name_len'] = df['name'].apply(len)`
`df[['A', 'B']].apply(np.sum, axis=0)`

Aggregation, Grouping and Reshaping

These are powerful tools for summarizing data and changing its structure for analysis.

- Basic Stats $\rightarrow$ `df.mean()`, `df.median()`, `df.sum()`, `df.min()`, `df.max()`, `df.count()`, `df.std()`

eg. `df['age'].mean()`

- `df.agg()` / `df.aggregate()` $\rightarrow$ Applies one or more aggregation functions across a specified axis.

eg. `df.agg({'age': ['mean', 'std'], 'Salary': 'sum'})`

- `df.groupby()` $\rightarrow$ Groups data using a mapper or by a Series of columns, allowing for split-apply-combine operations.

eg. `df.groupby('City')['Salary'].mean()` | `df.groupby(['City', 'gender']).std()`

- `df.pivot_table()` $\rightarrow$ Creates a spreadsheet-style pivot table as a DataFrame

eg. `pd.pivot_table(df, values='Sales', index='region', columns='Product', aggfunc='sum')`

- `pd.crosstab()` $\rightarrow$ Computes a simple cross-tabulation of two (or more) factors.

eg. `pd.crosstab(df['gender'], df['Smoker'])`

- `df.melt()` $\rightarrow$ Unpivots a DataFrame from wide format to long format.

eg. `pd.melt(df, id_vars=['id'], value_vars=['Product1', 'Product2'])`

• `pd.concat()` → Concatenates pandas objects along a particular axis.

eg. `pd.concat([df1, df2], axis=0)`

• `pd.merge()` / `df.merge()` → Merges Dataframe objects by performing a database-style join operation.

eg. `pd.merge(df-left, df-right, on='key', how='left')`

Working with Text, Dates and Time Series.

Pandas has specialized accessors (`.str`, `.dt`) and functions for handling these common data types.

• `df[col].str.lower()` → Converting strings in a Series to lowercase.

eg. `df[name].str.upper()`

• `df[col].str.contains()` → Tests if a pattern or regex is contained within each string.

eg. `df[address].str.contains('Ave')`

• `df[col].str.split()` → Splits strings around a given separator.

eg. `df[name].str.split(',', expand=True)`

• `df[col].str.replace()` → Replaces a pattern or regex.

eg. `df[phone].str.replace('-', '')`

• `pd.to_datetime()` → Converts a column to datetime objects.

eg. `df[date] = pd.to_datetime(df[date])`

• `df['col'].dt.year/` → Extract date components from a date time Series.
• `month/day`

eg. `df['year'] = df['date'].dt.year`

• `df.resample()` → Resamples time-series data to a new frequency (eg., from daily to monthly)

eg. `daily_data.resample('M').mean()`

Statistical and Utility Functions

Additional functions for more specific statistical tasks or utility operations.

• `df.corr()` → Computes pairwise correlation of columns (default is Pearson)

eg. `df[['age', 'salary']].corr()`

• `df.cov()` → Computes pairwise covariance of columns.

eg. `df[['age', 'salary']].cov()`

• `df.nunique()` → Counts the number of distinct/unique elements in each column.

eg. `df['city'].nunique()`

• `df.sample()` → Returns a random sample of items from an axis of the Dataframe.

eg. `df.sample(n=10)`

• `df.memory_usage()` → Returns the memory usage of each column in bytes.

eg. `df.memory_usage(deep=True)`